10.3 Architecture Decision Records

1. ADR 001

ADR-001	Contenido
Título	Selección del patrón de lógica de negocio
Estado	Aceptada
Contexto	`OrdenDeServicio` tiene 9 estados con transiciones condicionadas y más de 10 conceptos relacionados (Vehículo, Cliente, Diagnóstico, Cotización, Trabajo, Autorización, Refacción, Orden de Compra, Facturación Consolidada). El levantamiento documenta reglas transversales: umbral del 20% en ampliaciones (pregunta 12), escalamiento a 24h sin respuesta (pregunta 13), autorización parcial por trabajo (US-05/06) y trato diferenciado por tipo de cliente (BN-05). Se consideraron Transaction Script (rápido, pero repite cada regla en cada script), Table Module (resuelve consultas sobre conjuntos, pero no protege invariantes por instancia) y Domain Model (encapsula cada regla en el objeto que le corresponde)
Decisión	Domain Model como patrón principal para todo el flujo de servicio (OrdenDeServicio, Cotización, Trabajo, Autorización, Cliente/ClienteFlotilla). Table Module ligero, como excepción delimitada, únicamente para el módulo de reportes (US-15), donde no hay reglas de negocio que proteger, solo agregación de datos ya existentes
Consecuencias	Beneficio: las invariantes de estado y las reglas de autorización quedan protegidas en un solo lugar, sin repetirse en cada pantalla. Costo: mayor inversión de diseño inicial, porque hay que definir Aggregates, invariantes y Value Objects antes del primer caso de uso completo. Riesgo: el equipo debe evitar reducir las entidades a simples contenedores de datos. Trabajo posterior: documentar el mapa de dominio completo (Etapa 3) antes de construir la Service Layer
Trazabilidad	BN-06, US-03, US-04, US-08, pregunta 12 del levantamiento

2. ADR 002

ADR-002	Contenido
Título	Separar gestión de refacción sin existencia en dos servicios
Estado	Aceptada
Contexto	Un solo servicio (`GestionarRefaccionSinExistenciaService`) coordinaba en una misma operación dos Aggregates distintos: `OrdenDeCompra` (gestionado por Compras)

	y `OrdenDeServicio` (seguimiento del vehículo). Esto violaba SRP: un cambio en la política de compras obligaría a tocar una clase que también afecta el estado del vehículo frente al cliente, y viceversa (hallazgo #2, Etapa 7). Se consideró mantener un servicio único con transacción distribuida, frente a dos servicios independientes coordinados por evento
Decisión	Dos servicios de un solo Aggregate cada uno — `SolicitarRefaccionAProveedorService` y `MarcarEsperaDeRefaccionService`— coordinados por el Domain Event `RefaccionSinExistenciaDetectada`
Consecuencias	Beneficio: independencia real entre Compras y seguimiento del vehículo; cada Aggregate se modifica y persiste sin depender del otro. Costo: la consistencia entre "existe la orden de compra" y "el vehículo está en espera" pasa de ser inmediata (ACID) a eventual. Riesgo: si el mecanismo de eventos falla silenciosamente, un vehículo podría quedar "en espera" sin la orden de compra correspondiente. Trabajo posterior: definir una estrategia de reintento o reconciliación para el evento
Trazabilidad	BN-03, US-11, US-12, US-13, pregunta 9 del levantamiento

3. ADR 003

ADR-003	Contenido
Título	Autorización polimórfica en Cliente/ClienteFlotilla
Estado	Aceptada
Contexto	Existía un servicio aparte, `AutorizarComoResponsableDeFlotillaService`, que duplicaba casi por completo la lógica de `AutorizarTrabajosService` solo para cambiar quién podía autorizar. Esto violaba Liskov: `ClienteFlotilla` no actuaba como un verdadero sustituto de `Cliente`, sino como una ruta de código distinta (hallazgo #4, Etapa 7). Se consideró mantener servicios separados por tipo de cliente, frente a un solo servicio con un método polimórfico
Decisión	Un solo servicio (`AutorizarTrabajosService`) que delega "quién puede autorizar" a un método polimórfico `validarAutorizador(usuario)`, implementado de forma distinta en `Cliente` (valida al titular) y en `ClienteFlotilla` (valida al responsable de cuenta)
Consecuencias	Beneficio: un solo punto de entrada para "autorizar trabajos" en los tres casos de uso que lo necesitan (CU-01, CU-02, CU-05). Costo: ninguno

	significativo. Riesgo: si aparece un tercer tipo de cliente con reglas muy distintas, habrá que confirmar que el polimorfismo simple siga siendo suficiente. Trabajo posterior: ninguno pendiente para el alcance actual
Trazabilidad	BN-05, US-16, pregunta 8 del levantamiento

4. ADR 004

ADR-004	Contenido
Título	Política intercambiable para el umbral de autorización (20%)
Estado	Aceptada
Contexto	`ServicioDeAutorizacion` estaba planteado como una sola clase que acumularía un condicional por cada regla nueva de autorización. Esto violaba Open/Closed: cada regla nueva obligaría a modificar un método ya existente en vez de extender el sistema (hallazgo #3, Etapa 7). Se consideró un único método con condicionales crecientes, frente a una interfaz con una implementación por regla
Decisión	Interfaz `IPoliticaDeAutorizacion` con políticas intercambiables (`PoliticaAutorizacionEstandar`, `PoliticaAutorizacionAmpliacion`), seleccionada según el caso en vez de ramificada internamente
Consecuencias	Beneficio: una regla nueva se agrega como una clase nueva, sin modificar las políticas existentes. Costo: más clases pequeñas, lo que exige una convención clara de nombres. Riesgo: ninguno relevante detectado. Trabajo posterior: queda pendiente con el stakeholder si el umbral del 20% debe aplicar también a fallas adicionales pequeñas, no solo a ampliaciones sobre trabajos ya autorizados
Trazabilidad	Pregunta 12 del levantamiento; hallazgo 6 de la Etapa 3